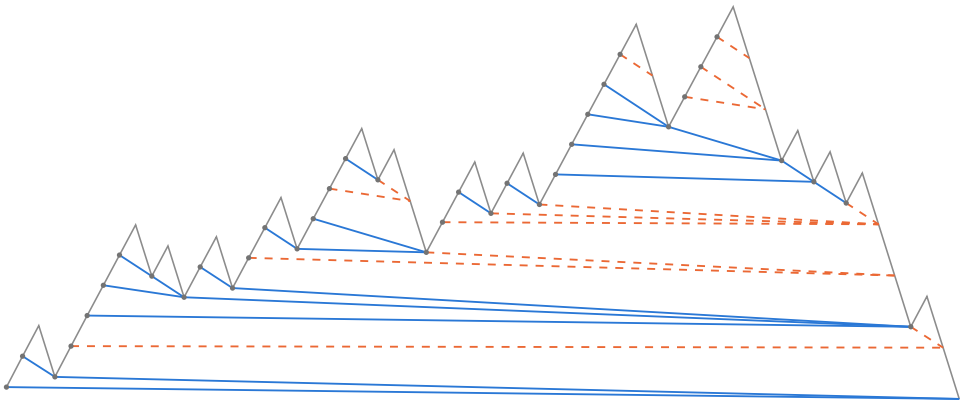
# Collatz, but stop at the drop

Now with less noise

**Ruud H.G. van Tol**
with assistance from Claude (Anthropic)

Draft, R_64b



© 2026 Ruud H.G. van Tol. Licensed under CC BY 4.0.

## How to read this paper

This paper is an introduction: it shows the rules of the game and leaves the proofs to the second paper [22], where most of them are checked by the proof assistant Lean 4 (Appendix B). You don't need any of that to follow the story: you can skip all references to Python and Lean, or look them up in the appendices. Names in sans-serif, like admissible_iff_chain, are signposts for readers who want to look up the proof. Examples can be replayed with the Python script `pv.py` (Appendix A); a command looks like `pv.py list 4`. At the end of each chapter, a box lists the key terms. The paper, the Lean proofs and the scripts were made with Claude (Anthropic) as a working partner; Appendix B tells how. Appendix C lists the first families.

## Acknowledgements

Mike Winkler pointed to the old probability idea behind the parts and checked the literature. Allan Wechsler read an early draft and suggested how to make it clearer.

# 1 The game

Pick a whole number, say 7. The *game* has two rules:

$$\textit{even}\text{: halve it;} \qquad \textit{odd}\text{: triple it, add one, and halve the result.}$$

(Three times an odd number plus one is always even, so the halving always works. In the OEIS these rules are A014682 [4].) Play from 7:

$$7 \to 11 \to 17 \to 26 \to 13 \to 20 \to 10 \to 5.$$

After seven steps we are at 5, below the 7 we started from. That is where the game ends: at the *drop*.

The famous version of this game, the Collatz conjecture, does not stop there: it asks whether every start eventually reaches 1. Nobody knows. Our game asks something that looks easier: does every start above 1 eventually get below itself? It is not really easier. If every number drops, a start drops to a smaller number, that one drops to a still smaller number, and so on, and whole numbers cannot keep getting smaller forever. So the chain of drops ends at 1. The other way round, a start above 1 that reaches 1 has dropped on the way. Both questions are the same question (tDrop_equiv_collatzT), and from here on we never play past a drop. The start 1 plays $1 \to 2 \to 1$, and we count that as its drop.

### Families that play alike

Play from 5, 9 and 13:

$$5 \to 8 \to 4, \qquad 9 \to 14 \to 7, \qquad 13 \to 20 \to 10.$$

Each is odd, then even, and after two steps it has dropped. The same holds for every fourth number from 1 on: $1, 5, 9, 13, 17, \ldots$ all play the same two steps. We call such numbers a *family*. Its members land on $1, 4, 7, 10, 13, \ldots$, every third number. Where they go next doesn't matter: they have dropped already. (How many factors 2 the numbers after the first step, $2, 8, 14, 20, \ldots$, hold is A375961 [18].)

Another family is $3, 19, 35, 51, \ldots$, every 16th number. Its members play odd, odd, even, even:

$$3 \to 5 \to 8 \to 4 \to 2, \qquad 19 \to 29 \to 44 \to 22 \to 11,$$

and land on $2, 11, 20, \ldots$, every 9th number.

The even numbers $2, 4, 6, \ldots$ are a family as well: one even step, and they have dropped.

Here are the first families (Table 1). In the last column, 1 stands for an odd step and 0 for an even one.

| family | | lands on | | steps |
|---|---|---|---|---|
| $2, 4, 6, \ldots$ | (every 2nd) | $1, 2, 3, \ldots$ | (every number) | 0 |
| $1, 5, 9, \ldots$ | (every 4th) | $1, 4, 7, \ldots$ | (every 3rd) | 10 |
| $3, 19, \ldots$ | (every 16th) | $2, 11, \ldots$ | (every 9th) | 1100 |
| $11, 43, \ldots$ | (every 32nd) | $10, 37, \ldots$ | (every 27th) | 11010 |
| $23, 55, \ldots$ | (every 32nd) | $20, 47, \ldots$ | (every 27th) | 11100 |
| $7, 135, \ldots$ | (every 128th) | $5, 86, \ldots$ | (every 81st) | 1110100 |
| $15, 143, \ldots$ | (every 128th) | $10, 91, \ldots$ | (every 81st) | 1111000 |
| $59, 187, \ldots$ | (every 128th) | $38, 119, \ldots$ | (every 81st) | 1101100 |

Table 1: The first families, by number of odd steps and lowest member: their members, where they land, and their steps.

Two things jump out. First, the gaps $2, 4, 16, 32, 128$ between family members turn into $1, 3, 9, 27, 81$: powers of two become powers of three. Follow two members side by side: an even step halves the gap between them, an odd step triples it and halves it. As long as the gap is still even, the two members are both odd or both even, and so they play the same steps. Second, the number of steps only depends on how many of them are odd: 1 odd step gives 2 steps, 2 give 4, 3 give 5, 4 give 7. Now write $3, 9, 27, 81$ in binary: $11_2$, $1001_2$, $11011_2$, $1010001_2$. They have 2, 4, 5 and 7 digits. Chapter 2 comes back to this.

So the game becomes: *does every number above* 1 *belong to some family?* That is still open (Chapter 7). On the way we find out:

- every family spells a word in an alphabet of two letters, and that word alone tells which family it is (Chapters 2 and 3);
- long words are made of shorter words, and which words fit together depends only on how many 1s they have (Chapter 4);
- that makes the words easy to count (Chapter 5);
- there is a faster way to play, and it sorts everything the same way (Chapter 6).

### The sequences in this story

The On-Line Encyclopedia of Integer Sequences (OEIS) knows many pieces of this game, each from its own angle. They all get a role here (Table 2).

| chapter | sequence | in this paper |
|---|---|---|
| 1 | A014682 | the rules of the game |
| 1 | A375961 | factors 2 in the number after the first step from $1, 5, 9, \ldots$ |
| 2 | A000265 | the odd part of a word |
| 2 | A020914 | the length of the valid words of each Order |
| 2 | A022921 | how much that length grows: 1 or 2 |
| 2, 5 | A100982 | the number of valid words of each Order |
| 2 | A368514 | the numbers in the counting grid |
| 3 | A177789 | the Collatz-primes of each Order |
| 3 | A243115 | all Collatz-primes of Order 2 and up, sorted |
| 3 | A122442 | the lowest Collatz-prime of each Order |
| 5 | A186008 | the Collatz-primes of each Order, as distances |
| 6 | A375782 | slides after a run of 1-stairs |
| 6 | A324245 | the index map of Vaillant and Delarue |
| 6 | A349414 | how much the index changes under that map |
| 6 | A387325 | head and length of the tail of 1s swapped |
| 7 | A076227 | counts behind the share of numbers not yet in a family |
| 7 | A060412 | the lowest floors not yet in a family |

Table 2: The OEIS sequences in this paper, by chapter; the words in the last column are explained in the chapter named.

**Key terms introduced in Chapter 1:**

| | |
|---|---|
| **the game** | even: halve; odd: triple, add one, halve. |
| **drop** | the first time the game gets below its start; for 1, the return to 1. |
| **family** | numbers with equal gaps that play the same steps up to their drop. |
| **A014682** | the rules of the game. |
| **A375961** | factors 2 in the number after the first step from $1, 5, 9, \ldots$ |

## 2 Two letters

Inside a family only the steps matter. So let us look at the steps on their own, without numbers.

The game has an alphabet of just two *letters*, `1` and `0`. Write `1` for a *1-step* (odd: triple, add one, halve) and `0` for a *0-step* (even: halve). Play a start until it drops, write down its steps, and you get a *word*: 123 spells `11011010`. As in any language, long words are made of shorter ones, and a hyphen shows the parts: `1-10-11010`. Chapter 4 shows where the hyphens go. The number of `1`s in a word is its *Order*: `11011010` has Order 5.

A word that a start spells on its way to its drop is a *valid word*. Which strings of `1`s and `0`s are valid, and how many are there?

Three rules hold for every valid word found so far. The second paper [22] proves them for the words of a *simpler rule*, the game with its "add one" left out and no floor needed. Keep two piles, both starting at 1: for every letter double the second pile, and for a `1` also triple the first. The word ends as soon as the second pile is the bigger one; `10` ends with the piles at 3 and 4. Whether every start spells exactly those words is the second of the two sharper questions of Chapter 7, checked for every Order up to two million.

- *A valid word ends with* `0`. A 1-step always goes up, so the drop is always a 0-step.
- *All valid words of the same Order have the same length*: the number of binary digits of $3^n$, for Order $n$, as Table 1 suggested. These lengths are $1, 2, 4, 5, 7, 8, 10, \ldots$ (A020914 [5]), and they grow by 1 or 2 at a time (A022921 [6]).
- *No valid word is the beginning of another one.*

The shortest valid word is `0`: start at 2, halve, done. Here are all seven valid words of Order 5, sorted as in a dictionary; the position in this list is a word's *rank*:

`11011010`, `11011100`, `11101010`, `11101100`, `11110010`, `11110100`, `11111000`.

The `0`s at the end of a valid word add nothing: the Order already fixes how many letters the word has. Read as a binary number, the word without them is its *odd part* (A000265 [3]): the number halved as long as it is even. Distinct valid words have distinct odd parts, and `pv.py` prints words as odd parts: `pv.py list 5` begins with `1101101`.

### Counting on a grid

There is a quick way to count valid words without listing them. A word is a walk on a grid: one step right for every `1`, one step down for every `0` (Figure 1). Write 1 at the top left, where every walk starts, and at every other point the number from the left plus the number from above, taking them from white circles only. The walk of a valid word ends at a blue circle; why they sit exactly there is explained in the second paper. The numbers in the blue circles count the valid words of each Order: $1, 1, 1, 2, 3, 7, 12$ for Orders 0 to 6, and then $30, 85, 173, \ldots$ (A100982 [9], from Order 1 on). The numbers in the white circles, column by column, are A368514 [16], which has a 0 at some of the blue circles. The grid never needs the words themselves, so it is fast: `pv.py count 60` prints the count for Order 60, a 25-digit number, at once.

Such a grid can do more than count. Filled in backwards, from the end of one Order, it tells at every point in how many ways a valid word of that Order can still be finished, and with those numbers the word at any rank can be written down directly. Go letter by letter: if the rank is at most the number of words that go on with 0, write 0; otherwise write 1 and take that number off the rank. So `pv.py unrank 5 4` prints 11101100, the fourth word of the list above, and for Order 60 the word two thirds of the way down the list comes just as fast, with no word before it ever listed. `pv.py rank` goes the other way.

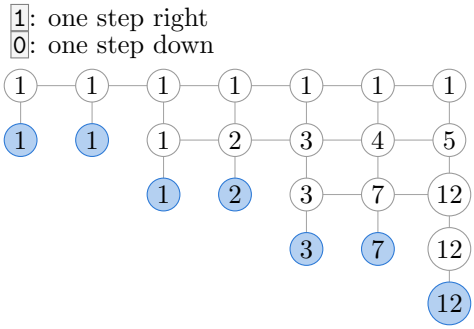



Figure 1: Counting walks. Each circle holds the number in the white circle to its left plus the number in the white circle above it; a blue circle then holds the number of valid words of one Order.

| **Key terms introduced in Chapter 2:** | |
|---|---|
| **letters** | 1 and 0, the whole alphabet. |
| **1-step, 0-step** | an odd step (triple, add one, halve); an even step (halve). |
| **word** | a string of letters; a hyphen shows its parts. |
| **Order** | the number of 1s in a word. |
| **valid word** | the word a start spells up to its drop. |
| **rank** | the position of a word in the dictionary list of its Order. |
| **odd part** | a word without its final 0s; as a binary number, halved while even (A000265). |
| **simpler rule** | two piles from 1: every letter doubles the second, a 1 also triples the first; the word ends when the second is bigger. |
| **A020914, A022921** | the lengths of valid words, and their steps 1 or 2. |
| **A100982** | the number of valid words of each Order. |
| **A368514** | the numbers in the counting grid. |

# 3 Every word is a family

Time to bring the numbers back, and with them a lift. A number is a *floor*, and the lift only ever stops at whole floors. Board at floor 123 and play the game: the lift stops at

$$185,\ 278,\ 139,\ 209,\ 314,\ 157,\ 236,\ 118.$$

Floor 118 is below 123, so the ride ends there: the drop of 123 is 118. The buttons pressed on the way, 1 for a 1-step and 0 for a 0-step, spell 11011010: a valid word, in parts 1-10-11010.

Seen from the side, the building has stairs (Figure 2). Every floor has exactly one way out: a pole down if it is even, a stair up if it is odd. An even floor has a hole with a fire pole, numbered 0: slide down to half the height, from floor 6 to floor 3, from 8 to 4, from 4 to 2. An odd floor has a stair numbered 1, a 1-stair, up: from floor 3 to floor 5, from 7 to 11. Stairs and poles run in closed shafts, with a door only at each end: once you step in, there is no getting off at a floor in between. Start at floor 3 and follow stairs and poles: 1 to floor 5, 1 to 8, 0 to 4, 0 to 2, below 3. The labels on the way spell the word 1100. The lift takes the same way, only without the climbing. Any building is too small: in the one of Figure 2, with 20 floors, the stairs from 15, 17 and 19 lead out through the roof. The page `building.html` that comes with this paper shows the building on screen: board at any floor and watch the ride to its drop.

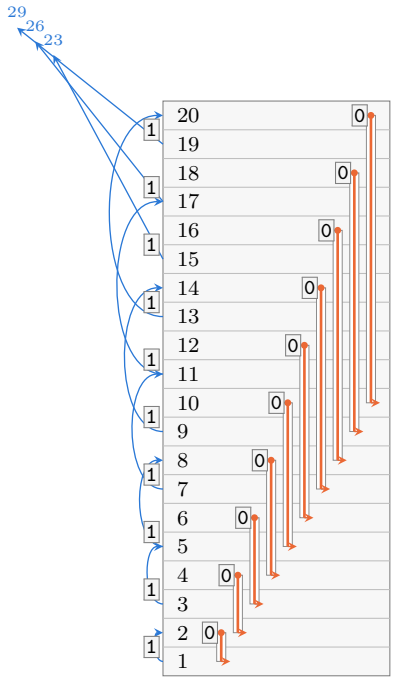



Figure 2: A building with 20 floors, from the side. Left, the stairs numbered 1 from the odd floors, up; inside, the fire poles numbered 0, each in a closed shaft from an even floor down to half its height. The stairs from 15, 17 and 19 lead to floors 23, 26 and 29, above the roof.

### Families of floors

Now board at floor $379 = 123 + 256$. The lift stops at $569, 854, 427, 641, 962, 481, 722$ and $361$: exactly the same buttons, and it lands on $361 = 118 + 243$. Board at $635 = 123 + 2 \cdot 256$ and it lands on $604 = 118 + 2 \cdot 243$. Every 256th floor from 123 on plays the same word, and the whole family lands on every 243rd floor from 118 on. Here $256 = 2^8$, one factor two for each of the 8 letters, and $243 = 3^5$, one factor three for each of the 5 ones: the gaps of Chapter 1 again.

Why every 256th floor? The first button only depends on whether the floor is odd or even: its last binary digit. The second button depends on the last two binary digits, the third on the last three, and so on: each letter of the word fixes one more binary digit of the floor. So a word of 8 letters picks exactly one floor out of every 256. Every word, valid or not, picks such a row of floors; for a valid word the row is its family (proved for the words of the simpler rule: tBits_classId, modEq_of_tBits_eq, classEquiv).

### Guess the family

This also works the other way round: make up a word first, and find its family afterwards. Start with the shortest word there is: 0. Which floors play it? A 0 means "even, halve": every even floor plays it, and has dropped after that one step. So 0 is a valid word, of Order 0, and it takes care of half of all floors.

Next, 10. The first letter asks for an odd floor. Of the odd floors, $1, 5, 9, \ldots$ go to an even floor next and $3, 7, 11, \ldots$ to an odd one, so 10 is the word of every 4th floor from 1. They all drop right at the end ($5 \to 8 \to 4$, and $1 \to 2 \to 1$ by our rule), so 10 is valid, of Order 1: another quarter. Together with 0, three quarters of all numbers are done. The word 1100 adds every 16th floor from 3, and so on: each valid word takes its own share, and no floor plays two words.

Now a longer one, 11010, of Order 3. Which floors play it? Find out one letter at a time; each letter doubles the gap, and leaves exactly two candidates, of which exactly one fits (Table 3).

| letters | candidates | | they play | fits |
|---|---|---|---|---|
| 1 | 1 or 2 | (every 2nd) | 1, 0 | 1 |
| 11 | 1 or 3 | (every 4th) | 10, 11 | 3 |
| 110 | 3 or 7 | (every 8th) | 110, 111 | 3 |
| 1101 | 3 or 11 | (every 16th) | 1100, 1101 | 11 |
| 11010 | 11 or 27 | (every 32nd) | 11010, 11011 | 11 |

Table 3: Guessing the family of 11010, one letter at a time.

So 11010 is the word of every 32nd floor from 11 on, and 11 is its lowest floor. No long ride was needed to find the family: five short ones, each as long as the letters so far. Is the word valid? Ride from the lowest floor: $11 \to 17 \to 26 \to 13 \to 20 \to 10$. The first floor below 11 comes exactly at the end of the word: valid.

### Collatz-primes

The lowest floor of the family of a valid word is its *Collatz-prime* (in Lean: its *class id*). Like a prime in a sieve, it strikes out its whole family at once; most Collatz-primes, such as 15 and 27, are no prime numbers. Each Order has as many Collatz-primes as valid words. Grouped by Order, they are the rows of A177789 [11]:

$$2 \mid 1 \mid 3 \mid 11, 23 \mid 7, 15, 59 \mid 39, 79, 95, 123, 175, 199, 219 \mid \ldots$$

Order 0 is the family of the even floors: one halving, and every even floor has dropped (A177789 has 0 where we write 2). Order 1 is the family $1, 5, 9, \ldots$ of Chapter 1. Merged and sorted from Order 2 on, the Collatz-primes are A243115 [13]: $3, 7, 11, 15, 23, 27, 31, 39, 47, 59, \ldots$. The lowest one of each Order, $2, 1, 3, 11, 7, 39, \ldots$, is A122442 [10] (which has 5 in the place of the 1: the lowest floor above 1).

A family can be written down without riding the lift at all: `pv.py attrs 1101` gives Order 3, Collatz-prime 11 and drop 10 for the word `11010` (the final `0`s are left off). The small "add one" of each 1-step decides where exactly the family lands; the gaps are always powers of two and three (tStep_iterate_shift).

| **Key terms introduced in Chapter 3:** | |
|---|---|
| **floor** | a number, as seen by the lift: it only stops at whole floors. |
| **family** | all floors that play the same valid word; for a word of $L$ letters, every $2^L$th floor. |
| **Collatz-prime** | the lowest floor of the family of a valid word; in Lean, the class id. |
| **A177789, A243115** | Collatz-primes by Order; all of Order 2 and up, sorted. |
| **A122442** | the lowest Collatz-prime of each Order. |

# 4 Words made of words

Where do the hyphens in 1-10-11010 come from? Ride from 123 again. After the first step, mark every floor that is lower than all floors before it, counted from the first step on: 185, 139 and 118. Cut the word there:

1-10-11010.

Each *part* after the first 1 is a valid word in its own right: 10 of Order 1, 11010 of Order 3. And for a reason: a part starts at a *new low* and ends at the next one, the first floor below where it started: its own drop. Cutting a walk at its lowest points is an old idea from probability [1], as M. Winkler pointed out; what it gives for valid words is new, as far as we know.

### Bars and a mountain

For every floor on the way, draw a bar to the first later floor below it: its own drop, a *sub-drop* (Figure 3). Floor 185 drops at 139, and the way there spells 10. Floor 139 drops at 118, and the way there spells 11010: the parts are the bars that run from one new low to the next.

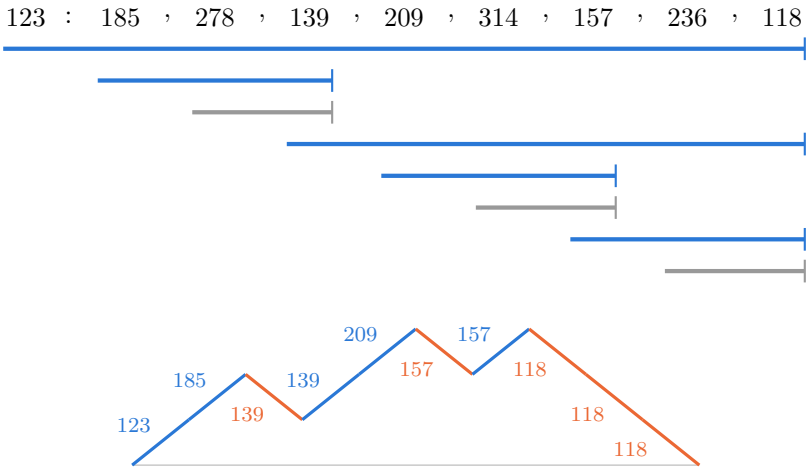



Figure 3: Top: the ride of 123, with a bar from every floor to its own drop; blue bars start at odd floors, grey ones at even floors. Bottom: the mountain of 123: one step up at every odd floor, one step down for each blue bar that ends.

The blue bars make a *mountain*. Every odd floor climbs one step; every time the lift stops below an earlier odd floor for the first time, it comes one step down for that floor. The mountain of 123 goes up, up, down, up, up, down, up, down, down, down: five steps up for the five 1s, five steps down, and it never goes below the ground. Every ride that reaches its drop makes such a mountain, because every bar ends by the drop at the latest. Figure 4 draws the bars of 27 over its ride.

*The bars belong to the word, not to the start.* Ride from 379, the next floor of the family of 123. The floors are different, 569, 854, 427, ..., down to 361, but every bar starts and ends at the same letters, so the hyphens and the mountain are the same too. The bars can even be drawn from the word alone, without riding at all. We checked this for every odd start up to 200 000: almost 600 000 bars, each found once by riding and once from the word alone, and they always agree. The second paper proves it for every ride, on one condition: that the second of the two sharper questions of Chapter 7 has the answer yes.

### The gluing rule

**Theorem 1** (Words made of words)**.** *Every valid word of Order at least* 1 *is, in exactly one way, a* `1`*, followed by zero or more valid parts of Order at least* 1*, followed by nothing or by one extra* `0`*. The other way round: glue a* `1` *and any valid parts together. Whether the result is a valid word depends only on the Orders of the parts, not on which parts they are.*

(Proved, as in Chapter 2, for the words of the simpler rule: exists_chain, chain_unique, admissible_iff_chain, validOrders_iff_dropLast, tail_eq_zero_iff.)

The *extra* `0` is needed when the last part ends just short of the drop. For example `1100` = `1`-`10`-`0`: floor 3 rides $3 \to 5 \to 8 \to 4 \to 2$. The part `10` is the ride of 5 to its drop 4, but 4 is not yet below 3; one more `0` brings it to 2. The exact rule for the Orders, and its proof, are in the second paper.

Here is the happy accident: *which* parts are used doesn't matter at all, only their Orders. The seven words of Order 5 show it (`pv.py list 5`, Table 4).

| Orders of the parts | Collatz-prime | word |
|---|---|---|
| $1, 3$ | 123 | `1`-`10`-`11010` |
| $1, 3$ | 219 | `1`-`10`-`11100` |
| $3, 1$ | 199 | `1`-`11010`-`10` |
| $3, 1$ | 79 | `1`-`11100`-`10` |
| 4 | 39 | `1`-`1101100` |
| 4 | 175 | `1`-`1110100` |
| 4 | 95 | `1`-`1111000` |

Table 4: The seven valid words of Order 5, cut into parts, grouped by the Orders of their parts.

Orders $1, 3$: one part of Order 1 times two of Order 3, two words. Orders $3, 1$: two words. Order 4: all three. The other five ways to split 4, such as $2, 2$ or $1, 1, 2$, never appear: a start would drop too early.

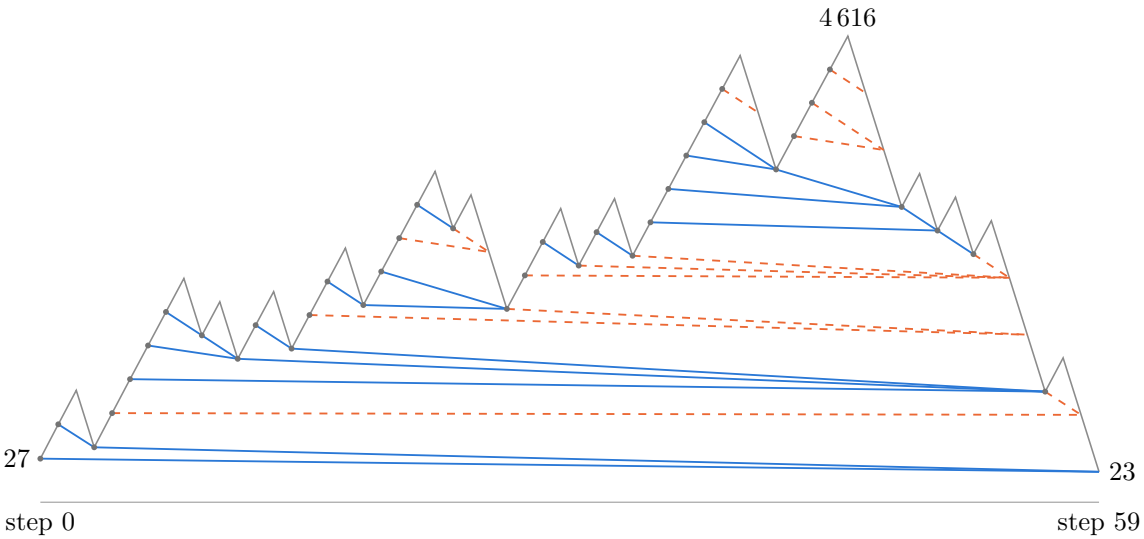



Figure 4: The ride of 27 to its drop at 23: 59 steps, up to floor 4 616 (the floors are drawn on a log scale). From every odd floor, a chord to its own drop: solid when the drop is an odd floor, dashed when it is an even one. These are the blue bars of Figure 3, drawn over the ride; they never cross.

**Key terms introduced in Chapter 4:**

| | |
|---|---|
| **new low** | a floor lower than all floors before it, from the first step on. |
| **part** | a piece between two new lows: a valid word of Order at least 1, unless it is the extra 0. |
| **extra 0** | the one 0 that may follow the last part. |
| **sub-drop** | the drop of a floor on the way; a bar in Figure 3. |
| **mountain** | up for every odd floor, down each time the ride first gets below that odd floor. |

# 5 How many words?

The counting grid of Chapter 2 gives the number of valid words of each Order:

$$1,\ 1,\ 1,\ 2,\ 3,\ 7,\ 12,\ 30,\ 85,\ 173,\ 476,\ 961,\ \ldots$$

for Orders $0, 1, 2, \ldots$ (A100982 [9], from Order 1 on). The gluing rule of Chapter 4 gives a second way to find them, and a third way tells roughly how big they get. As in Chapter 2, the proofs count the words of the simpler rule; the rides spell exactly those words as far as anyone has checked.

## Counting by gluing

A valid word is a $\boxed{1}$ followed by parts, and only the Orders of the parts matter. So list the allowed Orders, multiply the numbers of words for the parts, and add up. For Order 5 the parts after the first $\boxed{1}$ have Orders adding up to 4, and the allowed lists are $1, 3$ and $3, 1$ and 4:

$$1 \cdot 2 \ + \ 2 \cdot 1 \ + \ 3 \ = \ 7.$$

This *gluing count* works for every Order, with the numbers of all smaller Orders as the only ingredients (card_admSet_succ).

## About one in $n$

Take all words of the right length for Order $n$ that start with $\boxed{1}$, valid or not. For Order 5: length 8, a $\boxed{1}$ in front, and four more $\boxed{1}$s among the other seven letters: 35 words. Exactly 7 of them are valid, one in five. M. Winkler [23] proved that the number of valid words of Order $n$ is never more than one in $n$ of those words, and exactly one in $n$ at special Orders like $1, 3, 5, 17, 29, 41$. At those Orders the counts are *rational Catalan numbers*, cousins of the Catalan numbers that count mountains like the one in Figure 3.

## How often

Each valid word takes its own *share* of the floors (Chapter 3): a word of $L$ letters, one floor in every $2^L$. So the words of one Order together take a share of 1/2 for Order 0, 1/4 for Order 1, 1/16 for Order 2 and again $2/32 = 1/16$ for Order 3, then 3/128, 7/256, .... These are the shares of numbers whose drop takes $1, 2, 4, 5, 7, 8, \ldots$ steps: the pattern in A186008 [12].

| **Key terms introduced in Chapter 5:** | |
|---|---|
| **gluing count** | the number of words of an Order, from the numbers of words of the smaller Orders. |
| **rational Catalan** | a count of mountains under a slanted line; the counts at the special Orders. |
| **share** | the part of all floors that a word or an Order takes. |
| **A100982** | the number of valid words of each Order. |
| **A186008** | the Collatz-primes of each Order, as distances; their shares. |

# 6 A faster way to play

One day a very young engineer visits the building of Chapter 3. He counts the holes and poles and has an idea: why slide from 8 to 4, then from 4 to 2, then from 2 to 1, with three holes and three poles, when one *long pole* from 8 straight down to 1 does the same? And while he is at it, an *escalator*: from an odd floor, all the 1-stairs in a row in one go, together with the slide that follows them. From 27, which is $\texttt{11011}_2$, the escalator runs $27 \to 41 \to 62 \to 31$ in one ride.

His building works. The ride of 27 to its drop takes 59 steps; the *fast ride*, with escalators and long poles, takes 20:

$$27,\ 31,\ 121,\ 91,\ 103,\ 175,\ 445,\ 334,\ 167,\ 283,\ 319,$$
$$1822,\ 911,\ 2308,\ 577,\ 433,\ 325,\ 244,\ 61,\ 46,\ 23.$$

It climbs the same number of 1-stairs, so the Order stays the same (order_preservation).

### The accident

But the engineer forgot that the game ends at the drop (Figure 5). Board at floor 3: up the stairs to 8 ($3 \to 5 \to 8$), and the long pole from 8 goes straight down to 1, right past floor 2, where your game should have ended. In the game, the first floor below your start is a *free lunch*: everything after it is the game of a smaller start, which plays it on its own (Chapter 1). Sliding on wins nothing, and loses the one place where you were supposed to get off. So the fast ride has to remember where the drop was: on the last pole, you may have to get off halfway.

### A trick in binary

The escalator is a small trick on the binary digits of the floor. Take $91 = \texttt{1011011}_2$, a floor of the fast ride of 27. Cut off its last `0` and the `1`s after it (put a `0` in front if there is none): `011`, two `1`s. Multiply what is left, $\texttt{1011}_2 = 11$, by 3 for each cut `1`: $11 \cdot 9 = 99$. Add 1 for one cut `1`, 4 for two, 13 for three (that is `1`, `11`, `111` in base 3): $99 + 4 = 103$, the next floor of that fast ride. How many slides follow a run of 1-stairs is A375782 [17]. Vaillant and Delarue's related map [21] works on the index $n$ of the odd floor $2n + 1$ (A324245 [14]); how much that index changes is A349414 [15]. And just for fun: the same cut, at the last `0`, splits a number into a head and a tail of `1`s, and swapping the head with the length of the tail gives A387325 [19].

### Everything sorts alike

Write the fast ride of each Collatz-prime as a list: how many 1-stairs each escalator climbs, and how long each pole is, with a minus sign for a pole. Sort the Collatz-primes of one Order by their words, as in a dictionary, or by these lists, number by number: they come out sorted the same way (tBits_lt_iff_markedVector_lt, for Collatz-primes that drop at the end of their word). The families can even be grown sorted this way directly, like a tree where every word sprouts its possible next pieces (lex_enumeration); the rank of Chapter 2 is the place in this sorting.

| **Key terms introduced in Chapter 6:** | |
|---|---|
| **escalator** | all 1-stairs in a row at once, with the slide after them. |
| **long pole** | all slides in a row at once, down to an odd floor. |
| **fast ride** | the ride by escalators and long poles; it may pass the drop on its last pole. |
| **free lunch** | the drop: everything after it is the game of a smaller start. |
| **A375782, A349414** | slides after a run of 1-stairs; the index change under Vaillant and Delarue's map. |
| **A324245** | the index map of Vaillant and Delarue. |
| **A387325** | head and length of the tail of `1`s swapped. |

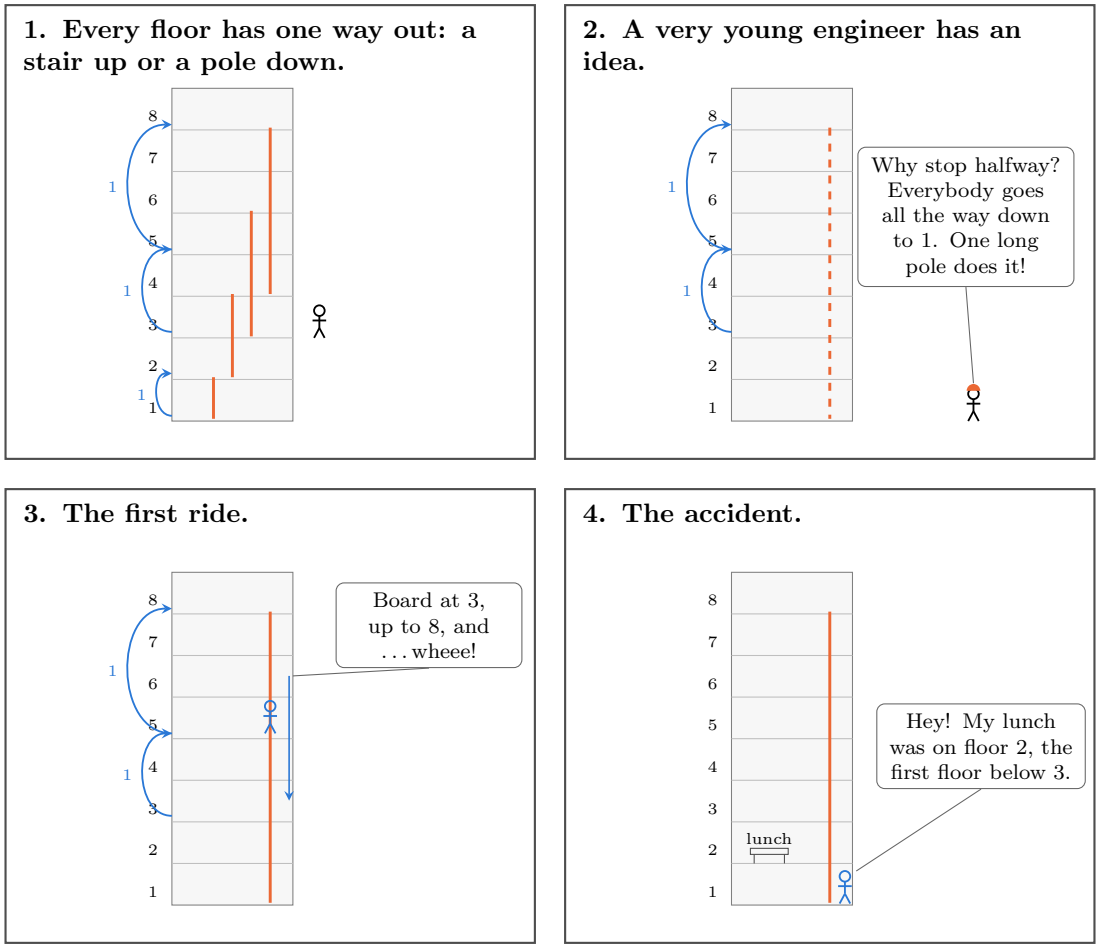



Figure 5: The young engineer and his long pole, all the way down to 1, as the famous version does.

# 7 What nobody knows yet

Here the game gets hard, for everybody. The *big question* is short: *does every floor above* 1 *drop?* Put in the words of this paper: does every floor above 1 belong to the family of some valid word? If the answer is yes, then by Chapter 1 the famous version is settled too. Nobody knows. The second paper splits the question into two sharper ones and shows what each would give. First: does every floor above 1 sooner or later spell a word of the simpler rule of Chapter 2? Second: does its ride then really drop at the end of that word, and not later?

### What we do know

- Every floor up to 984 572 779 224, almost $10^{12}$, drops: our own computer check, made for the second paper, went through them all. The closest call we know is 47: of the floors up to twenty million, it comes nearest to riding on past the end of its word, in a sense the second paper makes exact. Its word has Order 34, and its ride ends on floor 46.
- The share of numbers not yet in any family of Order at most $n$ shrinks: 1/2, 1/4, 3/16, 1/8, 13/128, 19/256, 1/16, ... for $n = 0, 1, 2, \ldots$ (A076227 [8], divided by powers of two). Almost every number drops: R. Terras proved that in 1976 [20].
- The lowest floor above 1 not yet in a family changes only now and then: 2, 3, 7, 27, 703, 10087, 35655, ... (A060412 [7]).

### Loops

Could a ride run in a *loop*? The ride $1 \to 2 \to 1$ does: it is the drop of 1. Any other loop would have a lowest floor, and that floor would never drop. So if every floor above 1 drops, there is no other loop. None is known.

### Two kinds of question

Which family a floor belongs to depends only on its last binary digits (Chapter 3): a question about digits, blind to size. Whether a floor drops at the end of that word depends on its size as well: a large floor does, and only for the smallest floors of a family can the "add one"s make the ride go on (the second sharper question). The big question sits exactly where these two worlds meet, and that is what makes it so hard [2].

| **Key terms introduced in Chapter 7:** | |
|---|---|
| **the big question** | does every floor above 1 drop? |
| **loop** | a ride that comes back to its start; only $1 \to 2 \to 1$ is known. |
| **A076227** | counts behind the share of numbers not yet in a family. |
| **A060412** | the lowest floors not yet in a family. |

# A pv.py

`pv.py` is a small Python script (Python 3.8 or later, no extra packages) that knows every valid word of every Order, without storing any lists. A word is typed as its *key*: the word without its final `0`s, so `1101` stands for `11010`. Table 5 lists the commands.

| command | result |
|---|---|
| `attrs KEY ...` | length, Order, Collatz-prime, drop, rank, parts |
| `count O ...` | the number of valid words of Order $O$ (A100982) |
| `list O [N]` | the valid words of Order $O$: the first $N$ (50 without $N$) |
| `rank KEY` / `unrank O R` | rank of a word / word of a rank |
| `test` | the self-test |

Table 5: The commands of `pv.py`.

A short session:

```
$ ./pv.py attrs 1101
Attrs(key='1101', p2=5, p3=3, x=11, drop=10, rank=1, rho=23,
      chain=['1', '1'], code=[(1, 1), (1, 1)])
$ ./pv.py count 5 20
5 7
20 5936673
```

Here `p2` is the length, `p3` the Order, `x` the Collatz-prime and `drop` where it lands, `rank` its rank, `rho` a number for the second paper; `chain` lists the keys of the parts, and `code` each part as (Order, rank). The script has more commands, for the second paper.

# B The Lean formalization

The Lean 4 project AcceleratedCollatz (with Mathlib) proves the results that the second paper cites by a Lean name, without gaps (what is still open enters as a condition); the sans-serif names in this paper point there. A self-contained file `ChainDecomposition.lean` holds the parts, the gluing rule and the counts, with Mathlib as its only dependency.

### How this was made

The Lean files, the scripts and this text were written in conversation with Claude (Anthropic): ideas were tried numerically first, then stated and proved in Lean, compiled locally, and every open end was logged for the next session. Some of it went better than planned. Once the words were cut at their lowest points, the parts turned out to be valid words again, glued by a rule on their Orders only; the proofs of Chapters 4 and 5 then became short and clean. An independent audit, commissioned by M. Winkler, compiled an earlier version of `ChainDecomposition.lean` and checked its proofs once more with a second checker. The same workflow is open to any reader: the Lean proofs can be checked on any computer, and `pv.py` replays the examples. Table 6 lists the main lean-files.

| lean-file | content |
|---|---|
| DropEquivalenceT | the drop game and the famous version are the same question |
| AdmissibleLength | valid words: length and beginnings |
| TCorrespondence, TAffine | every word is a family; where it lands |
| TChain{,Orders} | words made of words, the gluing rule |
| TCount, TBinomial | how many words |
| RMap, OrderPreservation | the fast ride |
| TLex, TVectorLex | everything sorts alike |

Table 6: The main lean-files; braces group names as in a shell, so TChain{,Orders} is TChain and TChainOrders.

The full list of lean-files is in the second paper.

# C The first families

Every family of Order 6 or less, and the first of Order 7, by rank: its floors (the Collatz-prime plus $i \cdot 2^L$, for $i = 0, 1, 2, \dots$), where they land, the fast ride of the Collatz-prime (Chapter 6), and the share of all floors not yet in any family listed so far (Table 7).

| Order | family $\to$ lands on | fast ride | left |
|---|---|---|---|
| 0 | $2+i{\cdot}2^1 \to 1+i{\cdot}3^0$ | $(2,1)$ | 1/2 |
| 1 | $1+i{\cdot}2^2 \to 1+i{\cdot}3^1$ | $(1,1)$ | 1/4 |
| 2 | $3+i{\cdot}2^4 \to 2+i{\cdot}3^2$ | $(3,4,2)$ | 3/16 |
| 3 | $11+i{\cdot}2^5 \to 10+i{\cdot}3^3$ | $(11,13,10)$ | 5/32 |
| 3 | $23+i{\cdot}2^5 \to 20+i{\cdot}3^3$ | $(23,40,20)$ | 1/8 |
| 4 | $59+i{\cdot}2^7 \to 38+i{\cdot}3^4$ | $(59,67,76,38)$ | 15/128 |
| 4 | $7+i{\cdot}2^7 \to 5+i{\cdot}3^4$ | $(7,13,10,5)$ | 7/64 |
| 4 | $15+i{\cdot}2^7 \to 10+i{\cdot}3^4$ | $(15,40,10)$ | 13/128 |
| 5 | $123+i{\cdot}2^8 \to 118+i{\cdot}3^5$ | $(123,139,157,118)$ | 25/256 |
| 5 | $219+i{\cdot}2^8 \to 209+i{\cdot}3^5$ | $(219,247,418,209)$ | 3/32 |
| 5 | $199+i{\cdot}2^8 \to 190+i{\cdot}3^5$ | $(199,337,253,190)$ | 23/256 |
| 5 | $39+i{\cdot}2^8 \to 38+i{\cdot}3^5$ | $(39,67,76,38)$ | 11/128 |
| 5 | $79+i{\cdot}2^8 \to 76+i{\cdot}3^5$ | $(79,202,101,76)$ | 21/256 |
| 5 | $175+i{\cdot}2^8 \to 167+i{\cdot}3^5$ | $(175,445,334,167)$ | 5/64 |
| 5 | $95+i{\cdot}2^8 \to 91+i{\cdot}3^5$ | $(95,364,91)$ | 19/256 |
| 6 | $507+i{\cdot}2^{10} \to 362+i{\cdot}3^6$ | $(507,571,643,724,362)$ | 75/1024 |
| 6 | $347+i{\cdot}2^{10} \to 248+i{\cdot}3^6$ | $(347,391,661,496,248)$ | 37/512 |
| 6 | $923+i{\cdot}2^{10} \to 658+i{\cdot}3^6$ | $(923,1039,2632,658)$ | 73/1024 |
| 6 | $583+i{\cdot}2^{10} \to 416+i{\cdot}3^6$ | $(583,985,739,832,416)$ | 9/128 |
| 6 | $423+i{\cdot}2^{10} \to 302+i{\cdot}3^6$ | $(423,715,805,604,302)$ | 71/1024 |
| 6 | $999+i{\cdot}2^{10} \to 712+i{\cdot}3^6$ | $(999,1687,2848,712)$ | 35/512 |
| 6 | $975+i{\cdot}2^{10} \to 695+i{\cdot}3^6$ | $(975,2470,1235,1390,695)$ | 69/1024 |
| 6 | $815+i{\cdot}2^{10} \to 581+i{\cdot}3^6$ | $(815,2065,1549,1162,581)$ | 17/256 |
| 6 | $367+i{\cdot}2^{10} \to 262+i{\cdot}3^6$ | $(367,931,1048,262)$ | 67/1024 |
| 6 | $735+i{\cdot}2^{10} \to 524+i{\cdot}3^6$ | $(735,2794,1397,1048,524)$ | 33/512 |
| 6 | $287+i{\cdot}2^{10} \to 205+i{\cdot}3^6$ | $(287,1093,820,205)$ | 65/1024 |
| 6 | $575+i{\cdot}2^{10} \to 410+i{\cdot}3^6$ | $(575,3280,410)$ | 1/16 |
| 7 | $1019+i{\cdot}2^{12} \to 545+i{\cdot}3^7$ | $(1019,1147,1291,1453,1090,545)$ | 255/4096 |

Table 7: The first families.

Each fast ride ends at the drop: where its last piece is a pole, you may have to get off halfway, as in $(3,4,2)$.

## References


[1] W. Feller, *An Introduction to Probability Theory and Its Applications*, Vol. II, 2nd ed., Wiley, 1971, Chapter XII.

[2] J. C. Lagarias, *The* $3x+1$ *problem and its generalizations*, Amer. Math. Monthly **92** (1985), 3–23.

[3] OEIS Foundation Inc., *Sequence A000265*, https://oeis.org/A000265.

[4] OEIS Foundation Inc., *Sequence A014682*, https://oeis.org/A014682.

[5] OEIS Foundation Inc., *Sequence A020914*, https://oeis.org/A020914.

[6] OEIS Foundation Inc., *Sequence A022921*, https://oeis.org/A022921.

[7] OEIS Foundation Inc., *Sequence A060412*, https://oeis.org/A060412.

[8] OEIS Foundation Inc., *Sequence A076227*, https://oeis.org/A076227.

[9] OEIS Foundation Inc., *Sequence A100982*, https://oeis.org/A100982.

[10] OEIS Foundation Inc., *Sequence A122442*, https://oeis.org/A122442.

[11] OEIS Foundation Inc., *Sequence A177789*, https://oeis.org/A177789.

[12] OEIS Foundation Inc., *Sequence A186008*, https://oeis.org/A186008.

[13] OEIS Foundation Inc., *Sequence A243115*, https://oeis.org/A243115.

[14] OEIS Foundation Inc., *Sequence A324245*, https://oeis.org/A324245.

[15] OEIS Foundation Inc., *Sequence A349414*, https://oeis.org/A349414.

[16] OEIS Foundation Inc., *Sequence A368514*, https://oeis.org/A368514.

[17] OEIS Foundation Inc., *Sequence A375782*, https://oeis.org/A375782.

[18] OEIS Foundation Inc., *Sequence A375961*, https://oeis.org/A375961.

[19] OEIS Foundation Inc., *Sequence A387325*, https://oeis.org/A387325.

[20] R. Terras, *A stopping time problem on the positive integers*, Acta Arith. **30** (1976), 241–252.

[21] N. Vaillant and P. Delarue, *The hidden face of the* $3x+1$ *problem. Part I: Intrinsic algorithm* (2019); linked from OEIS A324245.

[22] Ruud H.G. van Tol and M. Winkler, *Collatz, but stop at the drop; A game with two letters: the proofs*, draft R_64b (2026).

[23] M. Winkler, *Admissible* $qx+1$ *sequences, semiconvergents, and rational Catalan numbers*, arXiv:2609.22303 (2026).